

ProjectAAI628_model1

December 20, 2025

```
[1]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import random

import tensorflow as tf
from tensorflow.keras.optimizers import Adam
from sklearn.metrics import accuracy_score, precision_score, recall_score
from sklearn.model_selection import train_test_split
from tensorflow.keras import layers, losses, Model
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

from tensorflow.keras.models import Model

np.random.seed(42)
tf.random.set_seed(42)
random.seed(42)

[2]: # Download the dataset
dataframe = pd.read_csv('http://storage.googleapis.com/download.tensorflow.org/
↳data/ecg.csv', header=None)
raw_data = dataframe.values
dataframe.head()
```

```
[2]:
```

	0	1	2	3	4	5	6	\
0	-0.112522	-2.827204	-3.773897	-4.349751	-4.376041	-3.474986	-2.181408	
1	-1.100878	-3.996840	-4.285843	-4.506579	-4.022377	-3.234368	-1.566126	
2	-0.567088	-2.593450	-3.874230	-4.584095	-4.187449	-3.151462	-1.742940	
3	0.490473	-1.914407	-3.616364	-4.318823	-4.268016	-3.881110	-2.993280	
4	0.800232	-0.874252	-2.384761	-3.973292	-4.338224	-3.802422	-2.534510	

	7	8	9	...	131	132	133	134	\
0	-1.818286	-1.250522	-0.477492	...	0.792168	0.933541	0.796958	0.578621	
1	-0.992258	-0.754680	0.042321	...	0.538356	0.656881	0.787490	0.724046	
2	-1.490659	-1.183580	-0.394229	...	0.886073	0.531452	0.311377	-0.021919	
3	-1.671131	-1.333884	-0.965629	...	0.350816	0.499111	0.600345	0.842069	
4	-1.783423	-1.594450	-0.753199	...	1.148884	0.958434	1.059025	1.371682	

	135	136	137	138	139	140
0	0.257740	0.228077	0.123431	0.925286	0.193137	1.0
1	0.555784	0.476333	0.773820	1.119621	-1.436250	1.0
2	-0.713683	-0.532197	0.321097	0.904227	-0.421797	1.0
3	0.952074	0.990133	1.086798	1.403011	-0.383564	1.0
4	1.277392	0.960304	0.971020	1.614392	1.421456	1.0

[5 rows x 141 columns]

[]:

```
[3]: # The last element contains the labels
labels = 1 - raw_data[:, -1]

# The other data points are the electrocardriogram data
data = raw_data[:, 0:-1]

# data = data.reshape(-1, 140, 1)
# # labels = labels.reshape(-1, 140, 1)

labels.shape
```

[3]: (4998,)

[4]: data.shape

[4]: (4998, 140)

[5]: print(labels)

[0. 0. 0. ... 1. 1. 1.]

[6]:

[7]:

[8]:

```
[9]: train_data, test_data, train_labels, test_labels = train_test_split(
    data, labels, test_size=0.2, random_state=42, stratify=labels
)

min_val = tf.reduce_min(train_data)
max_val = tf.reduce_max(train_data)

train_data = (train_data - min_val) / (max_val - min_val)
```

```

test_data = (test_data - min_val) / (max_val - min_val)

train_data = tf.cast(train_data, tf.float32)
test_data = tf.cast(test_data, tf.float32)

train_labels = train_labels.astype(bool)
test_labels = test_labels.astype(bool)

normal_train_data = train_data[~train_labels]
normal_test_data = test_data[~test_labels]

anomalous_train_data = train_data[train_labels]
anomalous_test_data = test_data[test_labels]

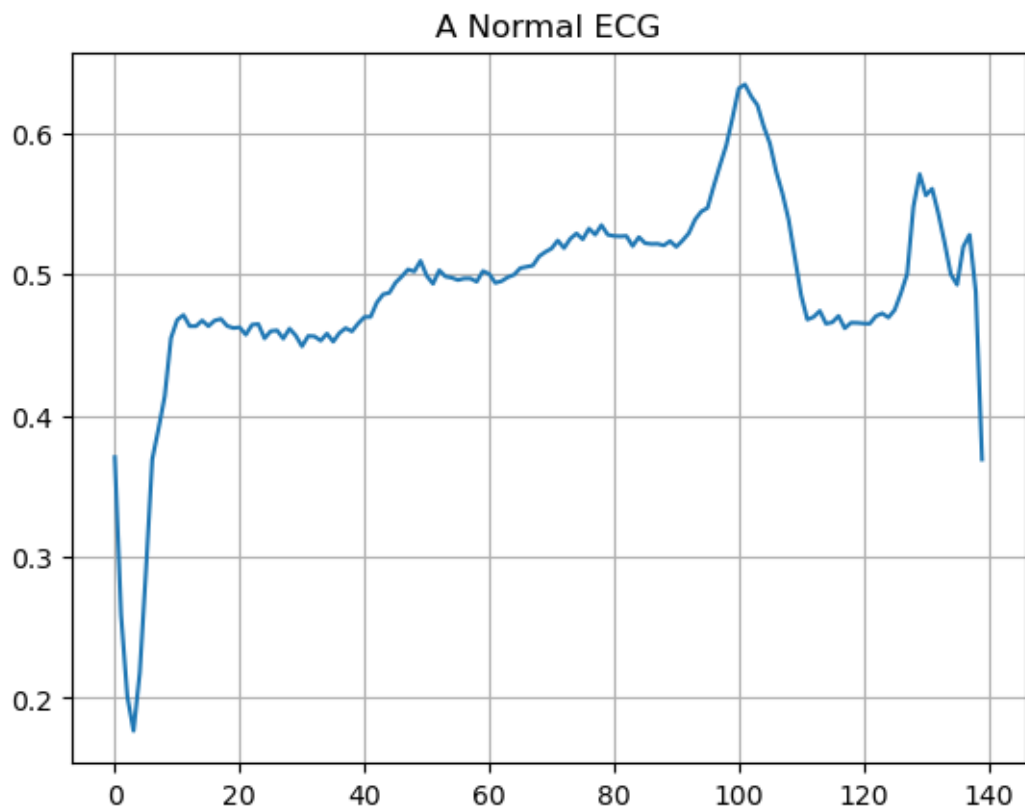
```

```
[ ]:
```

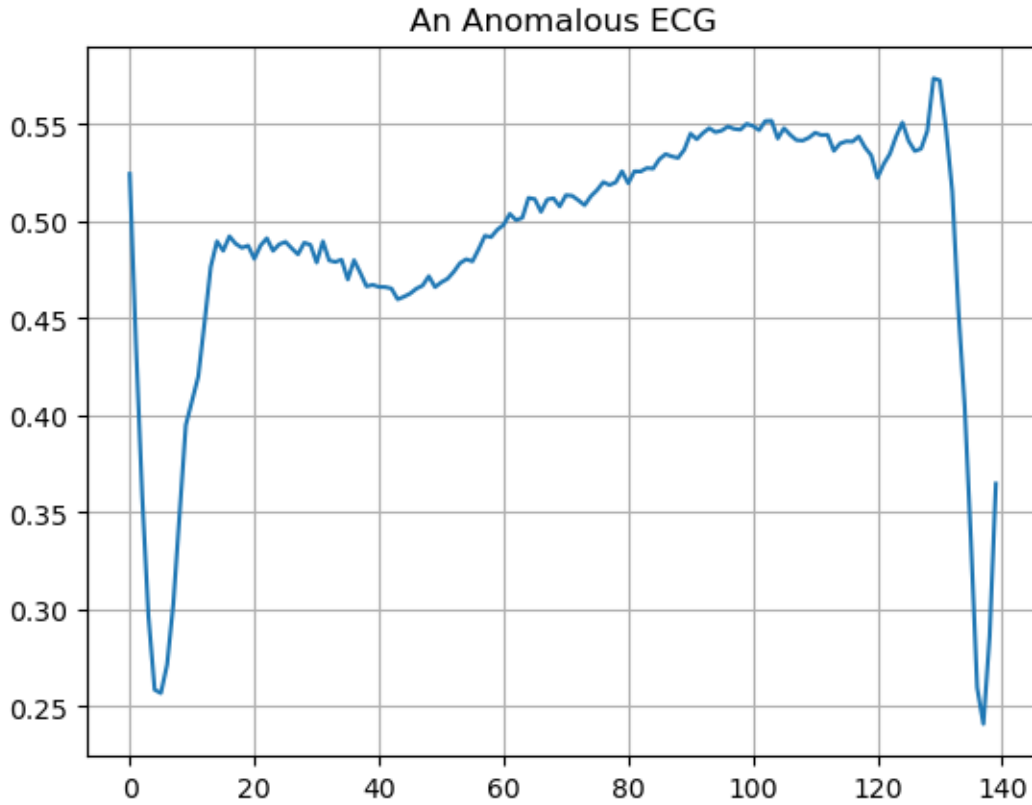
```

[10]: plt.grid()
plt.plot(np.arange(140), normal_train_data[0])
plt.title("A Normal ECG")
plt.show()

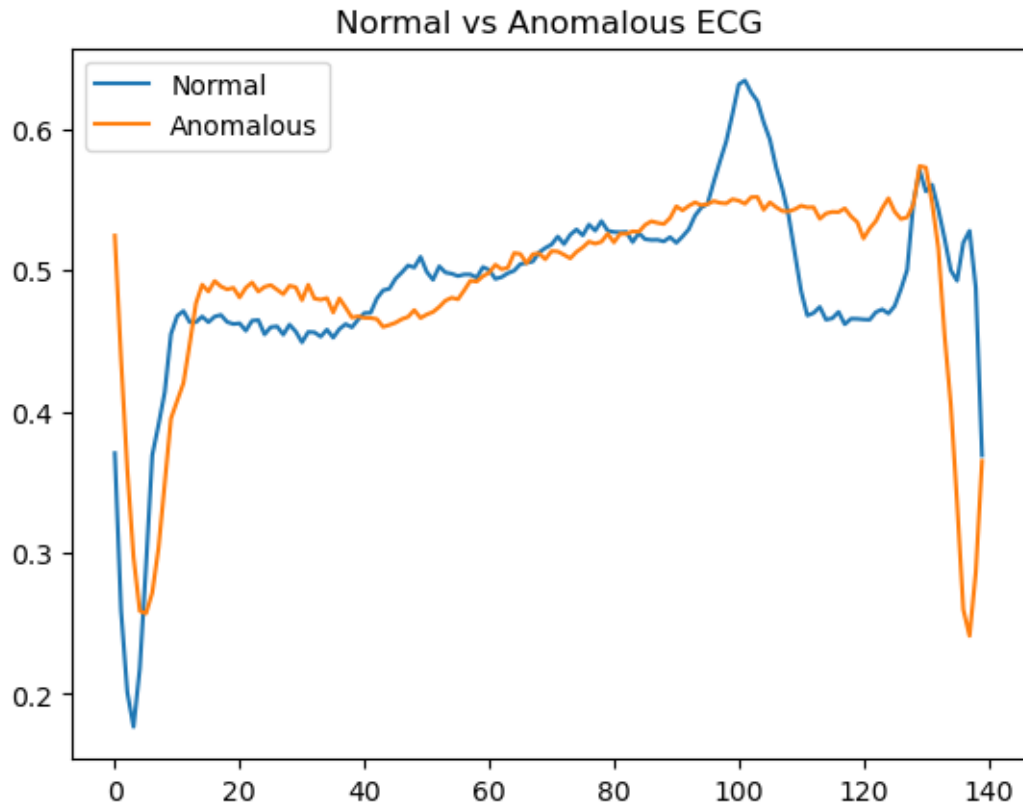
```



```
[11]: plt.grid()
plt.plot(np.arange(140), anomalous_train_data[0])
plt.title("An Anomalous ECG")
plt.show()
```



```
[12]: plt.plot(normal_train_data[0], label="Normal")
plt.plot(anomalous_train_data[0], label="Anomalous")
plt.legend()
plt.title("Normal vs Anomalous ECG")
plt.show()
```



[]:

```
[14]: ## CNN autoencoder Model 1
class CNNAnomalyDetector(Model):
    def __init__(self):
        super(CNNAnomalyDetector, self).__init__()

        # Encoder: Conv1D layers compress temporal features
        self.encoder = tf.keras.Sequential([
            layers.Conv1D(32, kernel_size=5, strides=1, activation="relu",
                padding="same"),
            layers.MaxPooling1D(pool_size=2, padding="same"),
            layers.Conv1D(64, kernel_size=5, strides=1, activation="relu",
                padding="same"),
            layers.MaxPooling1D(pool_size=2, padding="same"),
            layers.Conv1D(128, kernel_size=3, strides=1, activation="relu",
                padding="same"),
            layers.MaxPooling1D(pool_size=2, padding="same")
        ])

```

```

# Decoder: reconstruct signal using upsampling
self.decoder = tf.keras.Sequential([
    layers.Conv1D(128, 3, activation="relu", padding="same"),
    layers.UpSampling1D(2),
    layers.Conv1D(64, 5, activation="relu", padding="same"),
    layers.UpSampling1D(2),
    layers.Conv1D(32, 5, activation="relu", padding="same"),
    layers.UpSampling1D(2),
    layers.Conv1D(1, 3, activation="sigmoid", padding="same"),
    layers.Cropping1D(cropping=(2, 2))
])

def call(self, x):
    encoded = self.encoder(x)
    decoded = self.decoder(encoded)
    return decoded

# Build model
autoencoder = CNNAomalyDetector()
autoencoder.compile(optimizer="adam", loss="mae")
_ = autoencoder(tf.zeros((1, 140, 1)))

autoencoder.summary()

```

Model: "cnn_anomaly_detector"

Layer (type)	Output Shape	
Param #		
sequential (Sequential)	(1, 18, 128)	
↪35,200		
sequential_1 (Sequential)	(1, 140, 1)	
↪100,673		

Total params: 135,873 (530.75 KB)

Trainable params: 135,873 (530.75 KB)

Non-trainable params: 0 (0.00 B)

[]:

```
[16]: history = autoencoder.fit(normal_train_data, normal_train_data,  
                                epochs=20,  
                                batch_size=512,  
                                validation_data=(test_data, test_data),  
                                shuffle=True  
                                )
```

Epoch 1/20

5/5 1s 73ms/step - loss:
0.0456 - val_loss: 0.0439

Epoch 2/20

5/5 0s 32ms/step - loss:
0.0421 - val_loss: 0.0389

Epoch 3/20

5/5 0s 31ms/step - loss:
0.0389 - val_loss: 0.0368

Epoch 4/20

5/5 0s 32ms/step - loss:
0.0354 - val_loss: 0.0345

Epoch 5/20

5/5 0s 30ms/step - loss:
0.0326 - val_loss: 0.0326

Epoch 6/20

5/5 0s 32ms/step - loss:
0.0301 - val_loss: 0.0338

Epoch 7/20

5/5 0s 38ms/step - loss:
0.0284 - val_loss: 0.0284

Epoch 8/20

5/5 0s 31ms/step - loss:
0.0251 - val_loss: 0.0269

Epoch 9/20

5/5 0s 29ms/step - loss:
0.0265 - val_loss: 0.0366

Epoch 10/20

5/5 0s 30ms/step - loss:
0.0250 - val_loss: 0.0278

Epoch 11/20

5/5 0s 31ms/step - loss:
0.0210 - val_loss: 0.0256

Epoch 12/20

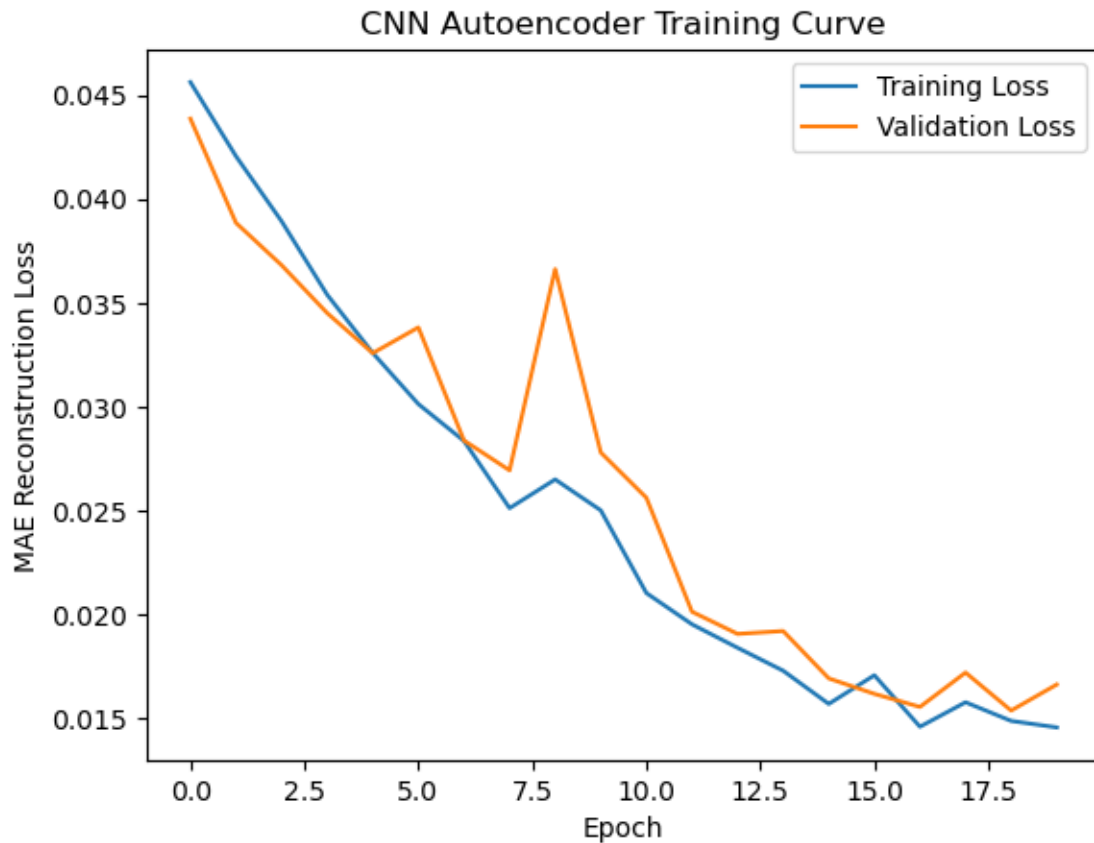
5/5 0s 30ms/step - loss:
0.0195 - val_loss: 0.0201

Epoch 13/20

5/5 0s 31ms/step - loss:
0.0184 - val_loss: 0.0191

```
Epoch 14/20
5/5          0s 30ms/step - loss:
0.0173 - val_loss: 0.0192
Epoch 15/20
5/5          0s 29ms/step - loss:
0.0157 - val_loss: 0.0169
Epoch 16/20
5/5          0s 29ms/step - loss:
0.0171 - val_loss: 0.0162
Epoch 17/20
5/5          0s 29ms/step - loss:
0.0146 - val_loss: 0.0155
Epoch 18/20
5/5          0s 30ms/step - loss:
0.0158 - val_loss: 0.0172
Epoch 19/20
5/5          0s 29ms/step - loss:
0.0149 - val_loss: 0.0154
Epoch 20/20
5/5          0s 30ms/step - loss:
0.0146 - val_loss: 0.0166
```

```
[17]: plt.plot(history.history["loss"], label="Training Loss")
      plt.plot(history.history["val_loss"], label="Validation Loss")
      plt.xlabel("Epoch")
      plt.ylabel("MAE Reconstruction Loss")
      plt.title("CNN Autoencoder Training Curve")
      plt.legend()
      plt.show()
```



```
[18]: encoded_data = autoencoder.encoder(normal_test_data).numpy()
      decoded_data = autoencoder.decoder(encoded_data).numpy()
```

```
(584, 18, 128)
```

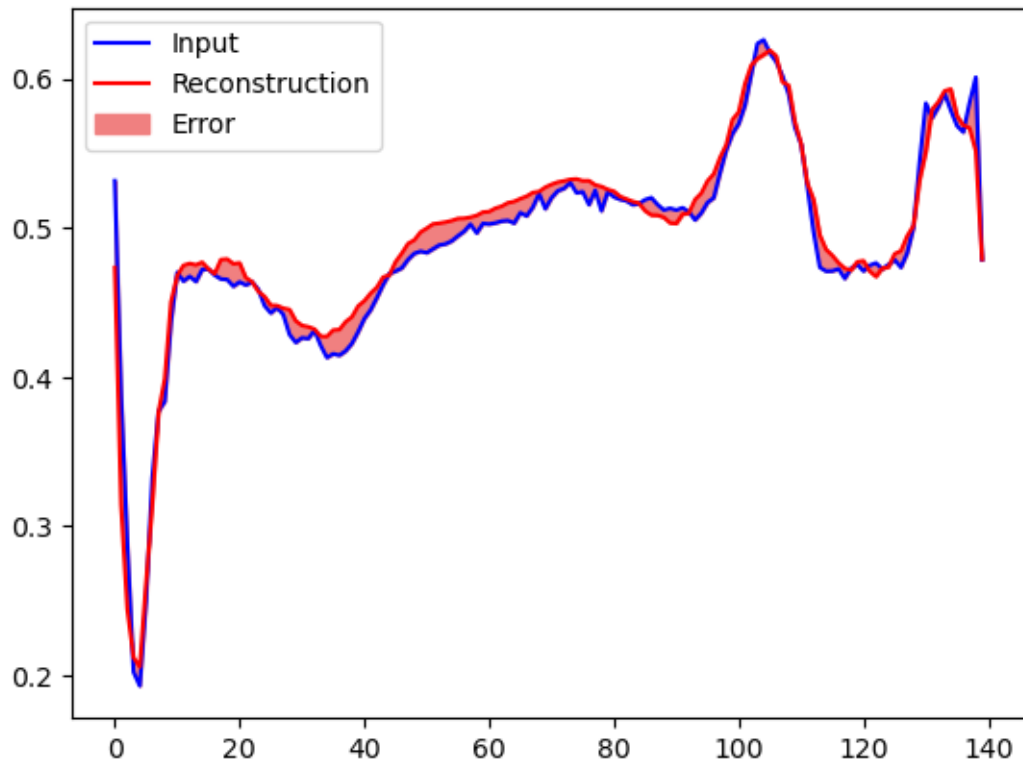
```
(584, 140, 1)
```

```
[ ]:
```

```
[19]: plt.plot(normal_test_data[0], 'b')
      plt.plot(decoded_data[0].squeeze(), 'r')

      plt.fill_between(
          np.arange(140),
          decoded_data[0].squeeze(),
          normal_test_data[0],
          color='lightcoral'
      )

      plt.legend(labels=["Input", "Reconstruction", "Error"])
      plt.show()
```



```
[20]: encoded_data = autoencoder.encoder(anomalous_test_data).numpy()
      decoded_data = autoencoder.decoder(encoded_data).numpy()
```

```
print(encoded_data.shape)
print(decoded_data.shape)
```

```
(416, 18, 128)
```

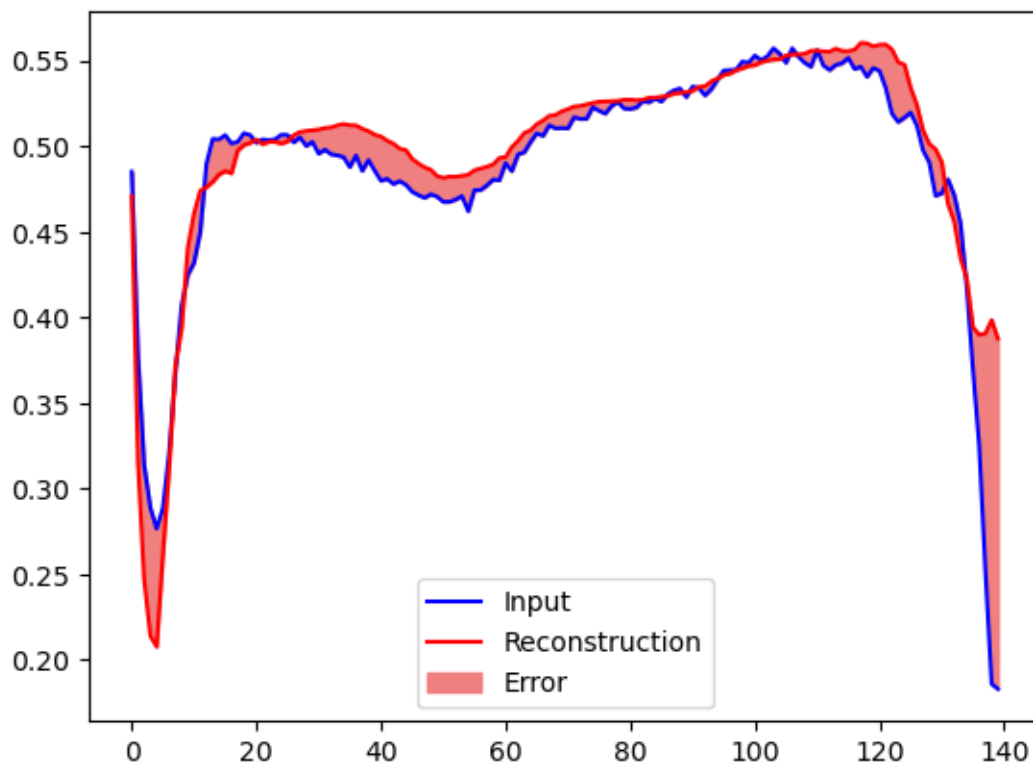
```
(416, 140, 1)
```

```
[21]: original = anomalous_test_data[0].numpy().squeeze()
      recon = decoded_data[0].squeeze()
```

```
plt.plot(original, 'b')
plt.plot(recon, 'r')
```

```
plt.fill_between(
    np.arange(140),
    recon,
    original,
    color='lightcoral'
)
```

```
plt.legend(labels=["Input", "Reconstruction", "Error"])
plt.show()
```



```
[ ]:
```

```
[22]: #Detect anomalies
```

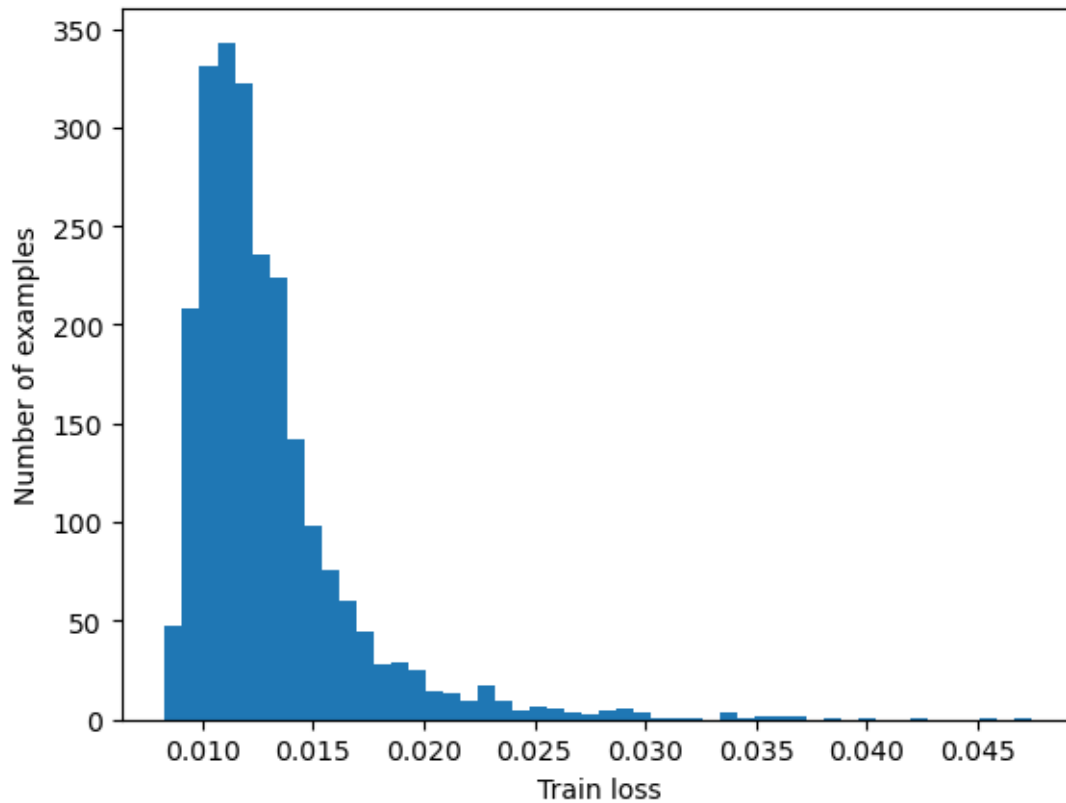
```
reconstructions = autoencoder.predict(normal_train_data)

train_loss = tf.reduce_mean(
    tf.abs(reconstructions.squeeze() - normal_train_data),
    axis=1
)

plt.hist(train_loss.numpy(), bins=50)
plt.xlabel("Train loss")
plt.ylabel("Number of examples")
plt.show()
```

73/73

0s 3ms/step



```
[23]: #threshold value
threshold = np.mean(train_loss) + np.std(train_loss)
print("Threshold: ", threshold)
```

Threshold: 0.0170426

```
[24]: from sklearn.metrics import precision_recall_curve

reconstructions = autoencoder.predict(data).squeeze()

test_loss = tf.reduce_mean(
    tf.abs(reconstructions - data),
    axis=1
).numpy()
```

157/157 0s 2ms/step

```
[ ]:
```

```
[25]: reconstructions = autoencoder.predict(anomalous_test_data).squeeze()

test_loss = tf.reduce_mean(
```

```

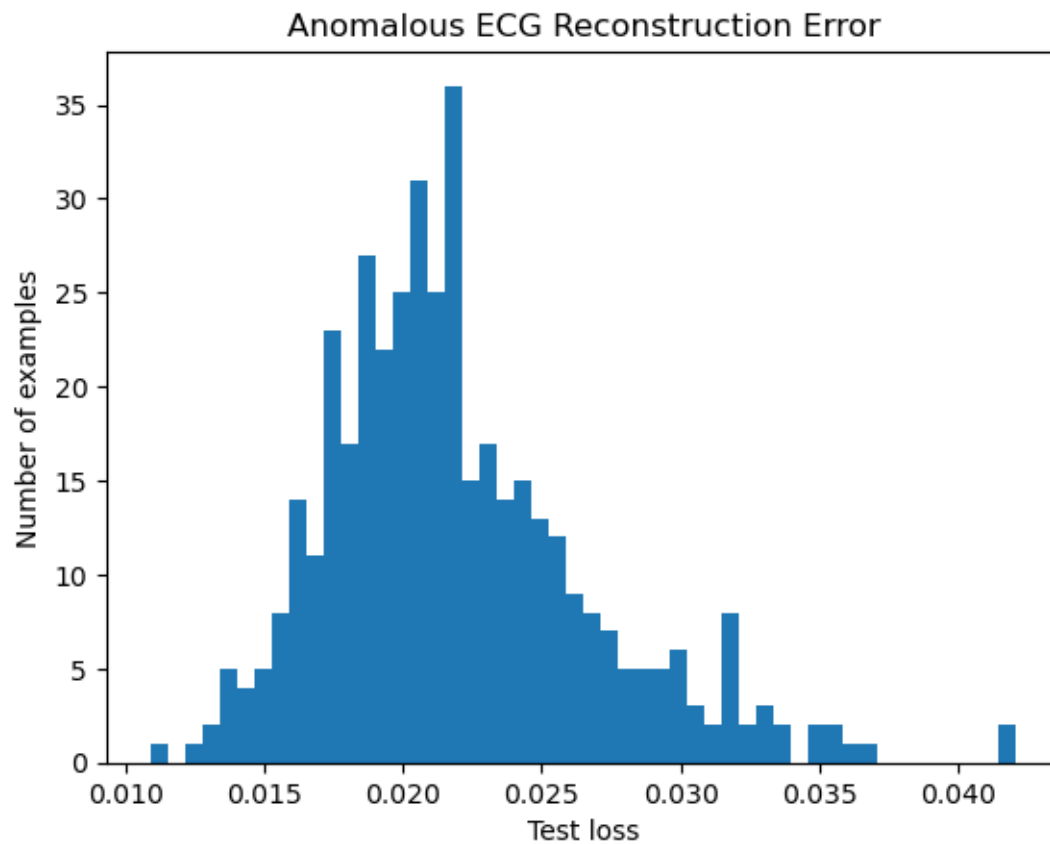
    tf.abs(reconstructions - anomalous_test_data),
    axis=1
).numpy()

plt.hist(test_loss, bins=50)
plt.xlabel("Test loss")
plt.ylabel("Number of examples")
plt.title("Anomalous ECG Reconstruction Error")
plt.show()

```

13/13

0s 2ms/step



[]:

```

[26]: def predict(model, data, threshold):
    reconstructions = model(data)

    # Ensure both tensors have the same shape
    reconstructions = tf.squeeze(reconstructions)
    data = tf.squeeze(data)

```

```

    loss = tf.reduce_mean(
        tf.abs(reconstructions - data),
        axis=1
    )

    return (loss > threshold).numpy().astype(int), loss.numpy()

```

```

[27]: def print_stats(predictions, labels):
        print("Accuracy = {}".format(accuracy_score(labels, predictions)))
        print("Precision = {}".format(precision_score(labels, predictions)))
        print("Recall = {}".format(recall_score(labels, predictions)))

```

```

[28]: #print(preds.shape, test_labels.shape)

```

```

[29]: preds, scores = predict(autoencoder, test_data, threshold)
        print_stats(preds, test_labels)

```

```

Accuracy = 0.901
Precision = 0.8800959232613909
Recall = 0.8822115384615384

```

```

[37]: from sklearn.metrics import confusion_matrix

```

```

[39]: import seaborn as sns

        # Ensure labels are integers
        test_labels = test_labels.astype(int)

        # Compute confusion matrix
        cm = confusion_matrix(test_labels, preds)

        # Accuracy
        accuracy = (cm[0,0] + cm[1,1]) / cm.sum() * 100

        # Percentages
        cm_percent = cm / cm.sum() * 100

        # Build annotation labels (same shape as cm)
        labels = np.empty_like(cm).astype(str)
        for i in range(cm.shape[0]):
            for j in range(cm.shape[1]):
                labels[i, j] = f"{cm[i, j]}\n({cm_percent[i, j]:.1f}%)"

        # Plot
        plt.figure(figsize=(5,4))
        sns.heatmap(cm, annot=labels, fmt="", cmap="Blues",

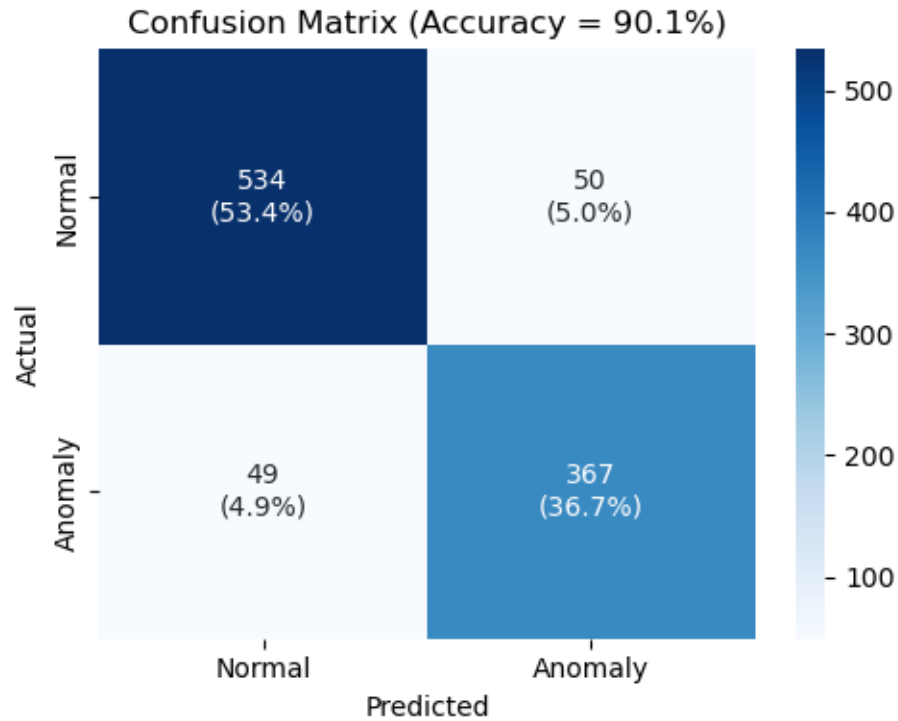
```

```

xticklabels=["Normal", "Anomaly"],
yticklabels=["Normal", "Anomaly"])

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title(f"Confusion Matrix (Accuracy = {accuracy:.1f}%)")
plt.tight_layout()
plt.show()

```



```

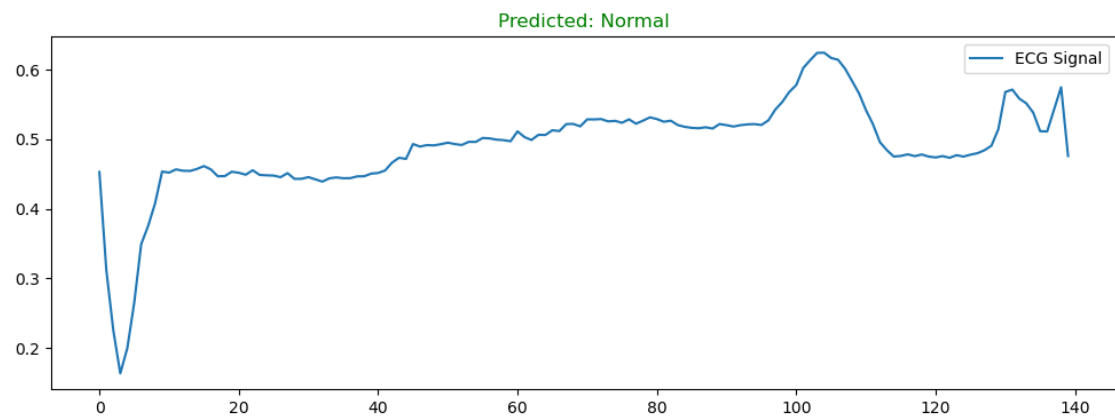
[42]: idx = 3 # choose a sample index 1 to 999

plt.figure(figsize=(12, 4))
plt.plot(test_data[idx], label="ECG Signal")

label = preds[idx]
color = "red" if label == 1 else "green"
plt.title(f"Predicted: {'Anomaly' if label==1 else 'Normal'}", color=color)

plt.legend()
plt.show()

```



[]: